



華南師範大學

本科学生实验（实践）报告

院 系：计算机学院

实验课程：编译原理实验

实验项目：实验 2：正则表达式转换 DFA 图

指导老师：黄煜廉

开课时间：2025 ~ 2026 年度第 1 学期

专 业：计算机科学与技术

班 级：计科 2 班

学生姓名：袁知本

华南师范大学教务处

华南师范大学实验报告

学生姓名 袁知本 学 号 20232131006
专 业 计算机科学与技术 年级、班级 2023 级 计科 2 班
课程名称 编译原理实验 实验项目 实验 2: 正则表达式转换为 DFA 图
实验类型 ☐验证 ☐设计 ☐综合 实验时间 2025 年 2 月 28 日
实验指导老师 黄煜廉 实验评分 _____

一、实验内容

设计一个应用软件,以实现将正则表达式-->NFA--->DFA-->DFA 最小化-->词法分析程序

1. 正则表达式应该可以支持命名操作, 运算符号有: 转义符号(\)、连接、选择(|)、闭包(*)、正闭包(+)、[], 可选(?), 括号()
2. 要提供一个源程序编辑界面, 让用户输入一行(一个)或多行(多个)正则表达式(可保存、打开正则表达式文件)
3. 需要提供窗口以便用户可以查看转换得到的 NFA (用状态转换表呈现即可)
4. 需要提供窗口以便用户可以查看转换得到的 DFA (用状态转换表呈现即可)
5. 需要提供窗口以便用户可以查看转换得到的最小化 DFA (用状态转换表呈现即可)
6. 要求应用程序为窗口式图形化界面
7. 将最小化得到 DFA 图转换得到的词法分析源程序(该分析程序需要用 C/C++ 语言描述, 而且只能采用讲稿中的转换方法一或转换方法二来生成源程序。), 系统需提供窗口以便用户可以查看转换得到的词法分析源程序(选做)
8. 书写完善的实验报告书, 书写格式请阅读文件: 实验报告书(含软件文档) 版头及正文格式.doc
9. 注意事项: 如果有完成选做内容, 在提交的实验材料中一定要有

关于这个内容的说明（选做）

二、实验目的

1. 实现一个带图形化界面的 Rust 源代码单词分类器
2. 要能识别并分类 Rust 源码中的标识符、关键字、字面量、注释、分隔符等单词标记，并将分类的结果和对应源码一并展示给用户
3. 深入了解词法分析的基本原理和实现方法，同时掌握开发图形化界面项目的要点
4. 熟练使用 C++的容器，利用 C++的容器的特性辅助词法分析的实现，同时理解正则表达式的运作原理

三、实验文档：

注：本次实验中的必做内容均已完成，同时也完成了全部的选做内容

1. 实验需求分析

本实验的目标是用 C++语言实现一个具有图形化界面的正则表达式转 NFA、DFA 图并进行 DFA 最小化，同时生成对应的词法分析程序的系统，本次实验的系统需要支持正则表达式的命名操作，并且支持诸如转义、闭包、正闭包、可选、或运算、连接运算等多种正则表达式运算符，而且，本实验的系统基于 MFC 图形界面开发，要打造成用户友好型系统，支持 NFA、DFA、最小化 DFA 状态转换图以及基于此生成的词法分析程序的动态可视化展示

功能需求

1) 用户界面设计

用户界面包括了状态栏、正则表达式输入区、有穷自动机状态转换图展示区和词法分析程序生成区共 4 个模块：

- 状态栏：显示提示信息，指导用户使用该系统；
- 正则表达式输入区：输入单行或多行正则表达式，点击【**转化为有穷自动机**】按钮后进行转换操作；
- 有穷自动机状态转换图展示区：借助 Graphviz 第三方包实现将转换关系绘制成图，以 bmp 格式的图片根据需求将

NFA、DFA 和最小化 DFA 的图显示在 picture control 控件中；

- 词法分析程序生成区：根据之前的最小化 DFA 转换关系，生成对应的词法分析程序，并返回显示在 MFC 图形化界面中，供用户拷贝使用

2) 正则表达式处理

支持对正则表达式的预处理，并添加显性连接符号，便于后续将正则表达式处理成有穷自动机图

3) 处理后的表达式进行中缀转后缀处理

根据正则表达式中的各种符号的运算优先级，通过栈处理将预处理的正则表达式转换为后缀形式，利于后续的 NFA、DFA、最小化 DFA 图的关系表构造

4) 生成 NFA 图

通过逐字符读取和栈处理，由最初的“状态——转换条件——>状态”子关系集不断往外构建扩大，最终生成完整的 NFA 图的关系表

5) 由 NFA 图生成 DFA 图

通过子集构造法将 NFA 转换生成 DFA，通过计算 NFA 图中的 ε 闭包，计算 DFA 状态的转移实现

6) DFA 图最小化

基于 Hopcroft 算法，将刚才得到的 DFA 图的关系表中的关系进行分区构建，之后迭代分割分区，去重移除孤立状态，实现 DFA 图最小化

7) 生成并显示 NFA、DFA、最小化 DFA 图

借助 Graphviz 第三方包，将转换关系写入 dot 文件，通过指令生成图片格式的文件，借由 MFC 的 picture control 控件将生成的图像展示出来

8) 生成对应的词法分析程序

根据最小化 DFA 图的关系表，转换得到词法分析程序的代码，

可以对对应的字符串进行匹配

2. 实验环境

硬件设备：基于 x64 的处理器，RAM16.0GB

软件设备：Windows11 家庭版，VS2022，MFC

3. 系统设计

1) 简洁方便的交互界面：

2) 正则表达式处理：

本功能包含以下部分：

- **显性连接符号的添加：**正则表达式中，连接符号是隐性的、不显示的，需要进行显性化处理以在后续操作中能够正确识别并进行连接运算
- **范围（中括号）运算处理：**将中括号中的值和中括号运算处理为或运算，预处理方便后续的中缀转后缀处理
- **多字符、转义字符处理：**通过条件判断和用 `vector` 对该类字符的位置进行定位，实现对多字符、转义字符等多种字符的处理

3) 处理后的表达式进行中缀转后缀处理：

根据正则表达式中的各种符号的运算优先级，构建一个运算符优先级 `map`，对经由正则表达式处理后的正则表达式进行中缀转后缀的进一步处理，方便后面利用 `stack` 进行 NFA 转换关系的构建

4) 生成 NFA 图：

将此前处理得到的后缀表达式逐字符读取，借助之前的多字符、转义字符定位 `vector`，确保不会在字符读取上出错，将读取到的转换条件字符构造出最小 NFA 子图后入栈，当读到正则表达式运算符时，根据情况将先前入栈的转换字符出栈，进行相应的关系构造，将构造完成的 NFA 子图再次入栈，以

此类推直到将完整的 NFA 图关系构造完成

5) 由 NFA 图生成 DFA 图:

根据之前生成的 NFA 图的关系表,通过子集构造法构建 DFA 图的关系表,通过计算 NFA 图中的 ε 闭包,计算 DFA 状态的转移,摘除 NFA 图的关系表中所有的 ε 转换,将通过 ε 转换得到的中间状态合并,实现将 NFA 转换为 DFA,得到 DFA 图的关系表

6) DFA 图最小化:

基于 Hopcroft 算法,将刚才得到的 DFA 图的关系表中的关系进行初始化分区,之后构建状态转移表,通过 Hopcroft 算法迭代划分分区,若某一个关系关联的前后两个状态分别处于不同的分区,则分配一个新的分区给它,迭代对每个分区进行分割,直至分割至最小为止,每个分区用一个状态进行代表(每个分区相当于最小化 DFA 图中的一个状态),区分接受状态和非接受状态,去掉重复的部分,并去除掉孤立出来的死状态(不与任何状态(包括自身)转换的状态),最终得到最小化的 DFA 图的关系表

7) 生成并显示 NFA、DFA、最小化 DFA 图:

借助 Graphviz 第三方包,将此前获得的 NFA、DFA、最小化 DFA 图对应的关系表写入 dot 文件中,标出初始状态和结束状态,将所有状态都写入到 dot 文件中,之后通过指令将 dot 文件转化为 bmp 文件和 svg 文件,借由 MFC 的 picture control 控件将生成的图像展示出来

8) 生成对应的词法分析程序:

根据最小化 DFA 图的关系表,借助 unordered_map 实现输入对应类型的字符串,匹配后输出其类型(若不满足匹配条件,则输出 error,并指出不匹配的位置)

4. 系统实现

4.1 用户界面设计

4.1.1 布局:

主窗口包含正则表达式输入、NFA、DFA、最小化 DFA 图片文件路径选择的文本输入区域，有多个按钮分别实现正则表达式处理、处理结果展示等功能，又有状态栏对用户进行使用上的提示

主窗口界面如下图所示:



图 4-1-1 主窗口控件布局

4.1.2 交互逻辑:

在【正则表达式】模块的输入框中输入正则表达式（单行或多行），点击【转化为有穷自动机】按钮，实现对正则表达式处理并转换为 NFA 图，同时，若正则表达式等号左侧的字符串的首字符是下滑线“_”，则在点击按钮生成 NFA 图后继续生成 DFA 图并将其最小化，示例如下:

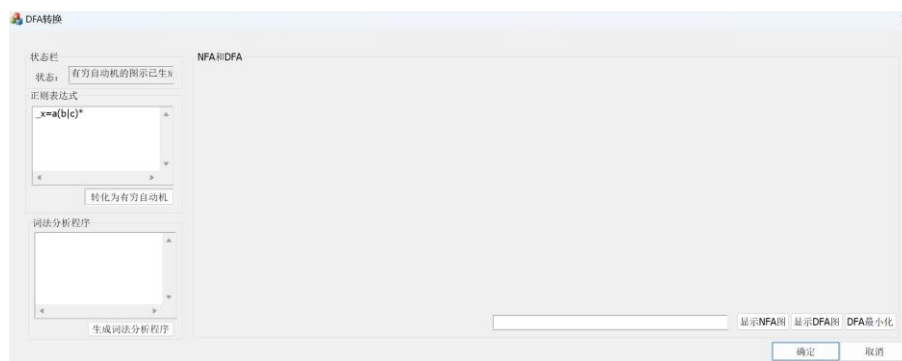


图 4-1-2 正则表达式处理功能展示

如上图所示，在处理完成后，状态栏会显示出“有穷自动机的图示已生成完成! bmp 格式图可以直接通过对应按钮打开查看，另外也可以在本地打开 svg 格式图”的提示，提高用户的交互友好性

在【NFA 和 DFA】展示模块中，在下方的输入栏中输入转换生成图片的名称，其命名格式如下:

- NFA 图: 正则表达式等号左侧纯字符串+NFA
- DFA 图: 正则表达式等号左侧纯字符串+DFA
- 最小化 DFA 图: 正则表达式等号左侧纯字符串+minDFA

以图 4-1-2 所示的正则表达式为例，则 NFA 图文件名为 xNFA，DFA

图文件名为 xDFA，最小化 DFA 图文件名为 xminDFA，其展示分别如下：

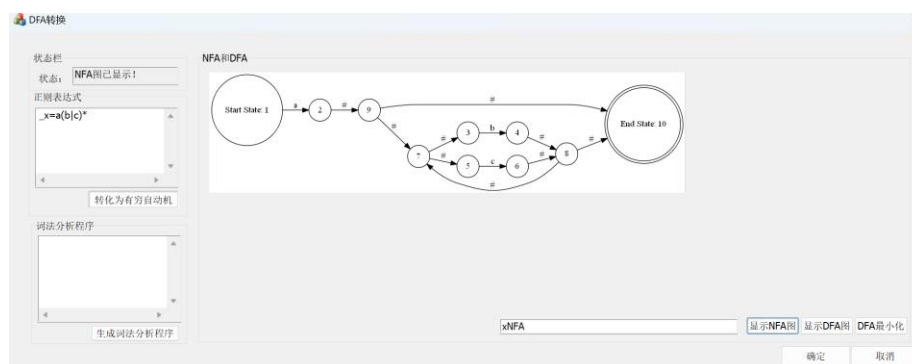


图 4-1-3 NFA 图展示功能演示

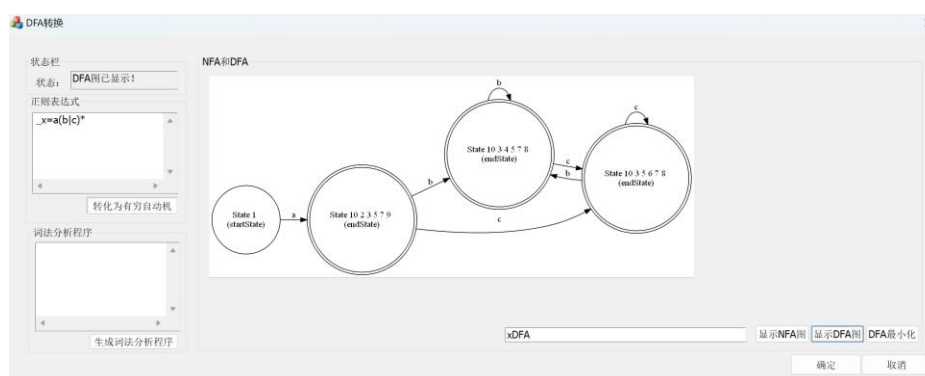


图 4-1-4 DFA 图展示功能演示

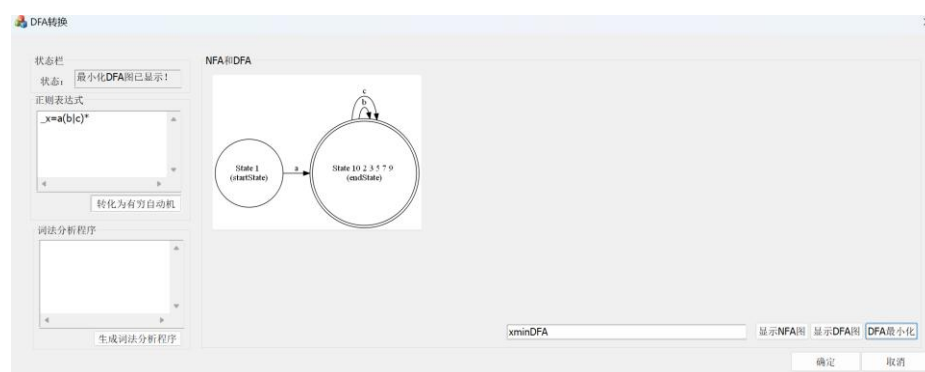


图 4-1-5 最小化 DFA 图展示功能演示

在做左下角的【词法分析程序】模块中，点击【生成词法分析程序】按钮即可获得词法分析程序的代码（基于最小化 DFA 图得到的字符串匹配代码）

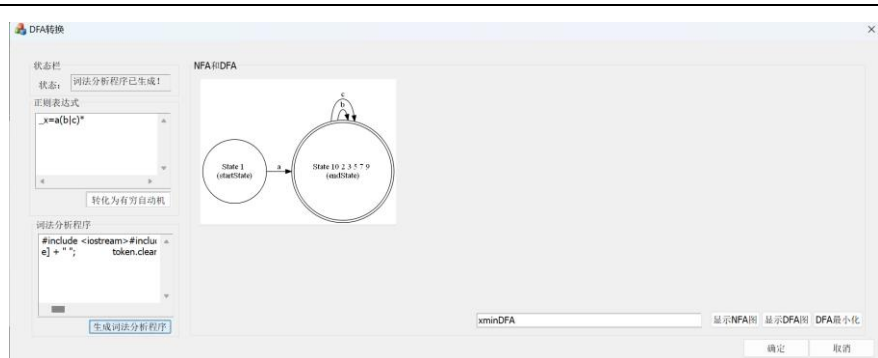


图 4-1-6 生成并输出词法分析程序功能演示

图 4-1-6 中的词法分析程序由于 MFC 的显示问题导致不能像文本编辑器中一样直观地查看到代码的全貌以及缩进情况，但实测过拷贝出来的格式是正确的，并且功能没有问题

4.1.3 用户交互界面的实现：

控件的实现：

如图 4-1-1 所示，用户交互界面的有按钮、文本编辑框和图片控制区等若干控件，均可通过 MFC 的工具箱拖拽进行添加，然后给文本编辑框和图片控制区设置对应的变量，对应的按钮设置响应函数，其代码如下图所示：

```
// 控件变量
void CCompilationprincipleexperiment2Dlg::DataExchange(CDataExchange* pDX)
{
    CDialogEx::DataExchange(pDX);
    DDX_Control(pDX, IDC_EDIT_REGULAR, m_REGULAR);
    DDX_Control(pDX, IDC_EDIT_PROGRAM, m_PROGRAM);
    DDX_Control(pDX, IDC_EDIT_STATUS, m_STATUS);
    DDX_Control(pDX, IDC_PIC_NFAandDFA, m_PIC);
    DDX_Control(pDX, IDC_EDIT_PATH, m_Path);
}

// 控件响应函数
BEGIN_MESSAGE_MAP(CCompilationprincipleexperiment2Dlg, CDialogEx)
    ON_WM_SYSCOMMAND()
    ON_WM_PAINT()
    ON_WM_QUERYDRAGICON()
    ON_BN_CLICKED(IDC_BUTTON_toNFAandDFA, &CCompilationprincipleexperiment2Dlg::OnBnClickedButtontonfaanddfa)
    ON_BN_CLICKED(IDC_BUTTON_MINDFA, &CCompilationprincipleexperiment2Dlg::OnBnClickedButtonMindfa)
    ON_BN_CLICKED(IDC_BUTTON_GENERATE, &CCompilationprincipleexperiment2Dlg::OnBnClickedButtonGenerate)
    ON_BN_CLICKED(IDC_BUTTON_NFA, &CCompilationprincipleexperiment2Dlg::OnBnClickedButtonNfa)
    ON_BN_CLICKED(IDC_BUTTON_DFA, &CCompilationprincipleexperiment2Dlg::OnBnClickedButtonDfa)
END_MESSAGE_MAP()
```

图 4-1-7 控件变量和响应函数一览

4.2 功能实现

4-2-1 正则表达式处理：

在用户点击【转化为有穷自动机】按钮后，程序将获取用户输入在文本编辑框中的正则表达式，先后进行预处理、中缀转后缀处理、转换

NFA、转换 DFA、DFA 最小化以及词法分析程序生成的操作，处理完成后，在状态栏显示提示语句告知用户

获取用户输入的正则表达式并进行处理的外部代码如下：

```
190 void CCompilationprincipleexperiment2Dlg::OnBnClickedButtonfaanddfa()
191 {
192     // TODO: 在此添加控件通知处理程序代码
193     // 正则表达式字符串获取
194     CString c_regular;
195     m_REGULAR.GetWindowTextW(c_regular);
196     string regular = CW2A(c_regular);
197
198     bool dfaflag = false;
199
200     regex.clear();
201     frontparts.clear();
202     backparts.clear();
203     endstates.clear();
204
205     // 正则表达式字符串处理（考虑多行和单行的情况）
206     string subre = "";
207     for (size_t i = 0; i < regular.size(); ++i)
208     {
209         subre += string(1, regular[i]);
210         if (regular[i + 1] == '\\r' && regular[i + 2] == '\\n' && i + 2 < regular.size())
211         {
212             regex.push_back(subre);
213             i += 2;
214             subre = "";
215         }
216     }
217     regex.push_back(subre);
218
219     // 针对每行正则表达式，按等号前后划分，前半截只取单词名，后半截取正则表达式
220     for (string item : regex)
221     {
222         int pos = item.find('=');
223         string fronttemp = item.substr(0, pos);
224         string backtemp = item.substr(pos + 1);
225         string frontpart = "";
226         for (char c : fronttemp)
227         {
228             if (!isalpha(c) && c != '_' && c != '_') break;
229             else if (c == '_' && c != '_')
230             {
231                 dfaflag = true;
232                 continue;
233             }
234             frontpart += c;
235         }
236         frontparts.push_back(frontpart);
237         backparts.push_back(backtemp);
238     }
239
240     // 处理多行正则表达式（每行单独处理）
241     for (size_t i = 0; i < regex.size(); ++i)
242     {
243         string regular = regex[i];
244         string re = regular.substr(regular.find('=') + 1);
245         vector<DFAState> dfastates;
246         vector<DFATransition> dfatransitions;
247         set<string> nfa_initial_state_set;
248         bool flag = false;
```

```

249 // 考虑当前正则表达式的符号是否为前若干行表达式对应的单词名
250 for (string str : frontparts)
251 {
252     if (re.find(str) != -1)
253     {
254         flag = true;
255         break;
256     }
257
258     string processed, postfix;
259     re = builder.opChange(re);
260     processed = builder.addConcatenation(re, frontparts);
261
262     postfix = builder.InfixtoPostfix(processed);
263
264     Elem NFA_elem = builder.buildNFA(postfix, frontparts);
265     builder.buildDFA(NFA_elem, dfastates, dfatransitions);
266     generateDotFile_NFA(NFA_elem, frontparts[i]);
267     if (dfaflag) generateDotFile_DFA(endstates, dfastates, dfatransitions, NFA_elem, frontparts[i]);
268 }
269
270 vector<DFAState> dfastates;
271 vector<DFATransition> dfatransitions;
272 string total_regex = regexReplace(frontparts, backparts, backparts.back());
273 total_regex = builder.opChange(total_regex);
274 string processed = builder.addConcatenation(total_regex, frontparts);
275 string postfix = builder.InfixtoPostfix(processed);
276 Elem NFA_elem = builder.buildNFA(postfix, frontparts);
277 builder.buildDFA(NFA_elem, dfastates, dfatransitions);
278 DFAbuilder::minimizeDFA(endstates, notendstates, dfatransitions);
279 code = code_generate(frontparts.back(), endstates, dfatransitions);
280
281 m_STATUS.SetWindowTextW(L"有穷自动机的图示已生成完成！bmp格式图可以直接通过对应按钮打开查看，另外也可以在本地打开svg格式图");

```

图 4-2-1、图 4-2-2 获取正则表达式并处理

预处理程序代码如下：

```

20 // 处理中括号，转换为或运算
21 string DFAbuilder::opChange(string re)
22 {
23     map<char, char> tmp;
24     string processed = "";
25     int i = 0, j = 0;
26     while (i < re.size())
27     {
28         if (re[i] == '[')
29         {
30             j = i;
31             while (re[j] != ']' && j < re.size()) ++j;
32             string sub_re = re.substr(i, j - i + 1);
33             for (int k = 0; k < sub_re.size(); ++k)
34             {
35                 if (sub_re[k] == '-' && k > 1 && k < sub_re.size() - 2)
36                     tmp.insert({ sub_re[k - 1], sub_re[k + 1] });
37             }
38             if (!tmp.empty())
39             {
40                 for (auto it : tmp)
41                 {
42                     string p = "(";
43                     p += it.first;
44                     for (int m = 0; m < int(it.second) - int(it.first); ++m)
45                         p += '|' + string(1, char(it.first + m + 1));
46                     processed += p + ')';
47                 }
48                 i = j + 1;
49             }
50             else
51             {
52                 string p = "(" + string(1, re[i + 1]);
53                 for (int n = i + 2; n < j; ++n)
54                     p += '|' + string(1, re[n]);
55                 processed += p + ')';
56                 i = j + 1;
57             }
58         }
59         if (i < re.size()) processed += re[i];
60         ++i;
61     }
62     return processed;
63 }

```

// 获取中括号的前后位置
// 考虑中括号内的数据之间有“-”的符号

// 考虑单字符

// 考虑多字符

// 将处理结果返回

```

65  ✓ string DFAbuilder::addConcatenation(string re, vector<string> subre_name)
66  {
67      string processed;
68      // 转义状态跟踪
69      bool escaped = false;
70
71      map<int, int> sub_pos;
72      if (subre_name.size() > 1)
73      {
74          ✓ for (string it : subre_name)
75          {
76              int last = 0;
77              string tmp = re;
78              ✓ while (tmp.find(it) != string::npos)
79              {
80                  sub_pos.insert({ last + tmp.find(it), it.size() });
81                  last = tmp.find(it) + it.size();
82                  tmp = tmp.substr(last);
83              }
84          }
85      }
86
87      size_t i = 0;
88
89      while (i < re.size())
90      {
91          bool subname_flag = false;
92          auto it = sub_pos.find(i);
93          if (it != sub_pos.end())
94          {
95              subname_flag = true;
96              string sub_re = re.substr(it->first, it->second);
97              processed += sub_re;
98              i += it->second;
99              --i;
100          }
101
102          if (!subname_flag) processed += re[i];
103          if (i + 1 < re.size())
104          {
105              // 判断当前字符是否为转义符
106              bool is_escape = (re[i] == '\\');
107
108              // 处理转义符逻辑
109              if (re[i] == '\\' && !escaped)
110              {
111                  escaped = true;
112                  ++i;
113                  continue;
114              }
115
116              if (!escaped && i + 1 < re.size())
117              {
118                  bool need_concat = false;
119
120                  // 左侧合法条件（扩展运算符范围）
121                  bool left_valid = (isalnum(re[i]) || re[i] == ')')
122                      || re[i] == '*' || re[i] == '+'
123                      || re[i] == '[' || re[i] == '{';
124
125                  // 右侧合法条件（支持更多运算符）
126                  bool right_valid = (isalnum(re[i + 1]) || re[i + 1] == '('
127                      || re[i + 1] == '[' || re[i + 1] == '\\'
128                      || re[i + 1] == '{');
129
130                  // 多字符运算符保护
131                  string part = string(1, re[i]) + string(1, re[i + 1]);
132                  if (find(symbol.begin(), symbol.end(), part) != symbol.end())
133                      need_concat = false;
134                  else
135                      need_concat = left_valid && right_valid;
136
137                  if (need_concat) processed += '.';
138              }
139
140              escaped = false;
141              ++i;
142          }
143          return processed;
144      }

```

图 4-2-3、图 4-2-4、图 4-2-5 预处理代码（用符号 ‘.’ 显性表示连接运算）

接下来，对预处理后的正则表达式进行中缀转后缀处理，这需要用到正则表达式运算优先顺序表，该表的定义如下：

```
11      // 运算优先级表
12      map<char, int> op_priority = {
13          {'*', 5}, {'+', 5},          // 闭包（闭包、正闭包）
14          {'?', 4},                  // 可选运算符
15          {'.', 3},                  // 显式连接符
16          {'|', 1},                  // 选择符
17          {'(', 0}                   // 括号特殊处理
18      };
```

图 4-2-6 正则表达式运算优先顺序表

由于在预处理的时候就已经处理过中括号，因此中括号不再出现在上图的运算优先级顺序表中

在借助该运算优先级表和栈后，我们可以逐一读取预处理后的正则表达式的字符，进行中缀转后缀的处理

中缀转后缀代码如下：

```

146 // 中缀转后缀表达式
147 string DfAbuilder::InfixtoPostfix(string processed)
148 {
149     string postfix, part = "";
150     stack<char> op_stack;
151     size_t i = 0;
152     while (i < processed.size())
153     {
154         char c = processed[i];
155         if (isalnum(c))
156         {
157             postfix += c; // 处理操作数（直接输出）
158             ++i;
159         }
160         else if (c == '\\')
161         {
162             if (i + 1 < processed.size())
163             {
164                 char escaped_char = processed[i + 1];
165                 postfix += string(1, c) + string(1, escaped_char);
166                 i += 2;
167             }
168         }
169         else if (c == '(')
170         {
171             op_stack.push(c); // 处理左括号（入栈）
172             ++i;
173         }
174         else if (c == ')') // 处理右括号（部分出栈）
175         {
176             while (!op_stack.empty() && op_stack.top() != '(')
177             {
178                 postfix += op_stack.top();
179                 op_stack.pop();
180             }
181             op_stack.pop();
182             ++i;
183         }
184         else if (c == '*' || c == '?' || c == '+' || c == '.' || c == '|')
185         {
186             while (!op_stack.empty() && op_priority[c] <= op_priority[op_stack.top()])
187             {
188                 postfix += op_stack.top();
189                 op_stack.pop();
190             }
191             op_stack.push(c);
192             ++i;
193         }
194         else
195         {
196             part = c + string(1, processed[i + 1]) + string(1, processed[i + 2]);
197             if (find(symbol.begin(), symbol.end(), part) != symbol.end())
198             {
199                 postfix += part;
200                 i += 3;
201             }
202             else
203             {
204                 part = c + string(1, processed[i + 1]);
205                 if (find(symbol.begin(), symbol.end(), part) != symbol.end())
206                 {
207                     postfix += part;
208                     i += 2;
209                 }
210                 else
211                 {
212                     postfix += c;
213                     ++i;
214                 }
215             }
216             part = "";
217         }
218     }
219     while (!op_stack.empty()) // 处理栈内剩余的运算符
220     {
221         postfix += op_stack.top();
222         op_stack.pop();
223     }
224     return postfix;

```

图 4-2-7、图 4-2-8 中缀转后缀代码

在完成对正则表达式的中缀转后缀处理后，处理的结果已经可以用

于构建 NFA 图的关系了，借助栈的功能，我们可以实现逐一构建 NFA 子图，在读取到运算符时，根据运算符的种类对前面的子图进行合并，用 ϵ 转换连接各状态，最终完成 NFA 图的构建：

```
227 // 创建新的状态
228 Node DFAbuilder::new_node()
229 {
230     Node newNode;
231     newNode.nodename = to_string(++nodenum);
232     return newNode;
233 }
234
235 // 拷贝NFA子图（用于NFA子图合并）
236 void DFAbuilder::elem_copy(Elem& dest, Elem source)
237 {
238     for (int i = 0; i < source.edgeCount; ++i)
239         dest.edgeSet.push_back(source.edgeSet[i]);
240     dest.edgeCount += source.edgeCount;
241 }
242
243 // 处理普通字符
244 Elem DFAbuilder::act_Elem(string c)
245 {
246     // 创建新节点（对应新的初态和终态）
247     Node startnode = new_node();
248     Node endnode = new_node();
249
250     // 创建新边
251     Edge newedge;
252     newedge.startname = startnode.nodename;
253     newedge.endname = endnode.nodename;
254     newedge.tranSymbol = c;
255
256     // 新NFA组成元素
257     Elem newelem;
258     newelem.edgeCount = 1;
259     newelem.edgeSet.push_back(newedge);
260     newelem.startname = newelem.edgeSet[0].startname;
261     newelem.endname = newelem.edgeSet[0].endname;
262
263     return newelem;
264 }
```

```

266     // 处理或运算
267     ✓Elem DFAbuilder::act_Unit(Elem fir, Elem sec)
268     {
269         Elem newelem;
270         newelem.edgeCount = 0;
271         Edge edge1, edge2, edge3, edge4;
272
273         // 获得新的状态节点
274         Node startnode = new_node();
275         Node endnode = new_node();
276
277         // 构建e1 (连接起点和AB的起始点A)
278         edge1.startname = startnode.nodename;
279         edge1.endname = fir.startname;
280         edge1.tranSymbol = "#";
281
282         // 构建e2 (连接起点和AB的起始点A)
283         edge2.startname = startnode.nodename;
284         edge2.endname = sec.startname;
285         edge2.tranSymbol = "#";
286
287         // 构建e3 (连接起点和AB的起始点A)
288         edge3.startname = fir.endname;
289         edge3.endname = endnode.nodename;
290         edge3.tranSymbol = "#";
291
292         // 构建e4 (连接起点和AB的起始点A)
293         edge4.startname = sec.endname;
294         edge4.endname = endnode.nodename;
295         edge4.tranSymbol = "#";
296
297         // 合并fir和sec
298         elem_copy(newelem, fir);
299         elem_copy(newelem, sec);
300
301         // 新构建四条边
302         newelem.edgeSet.push_back(edge1);
303         newelem.edgeSet.push_back(edge2);
304         newelem.edgeSet.push_back(edge3);
305         newelem.edgeSet.push_back(edge4);
306
307         newelem.edgeCount += 4;
308         newelem.startname = startnode.nodename;
309         newelem.endname = endnode.nodename;
310
311         return newelem;
312     }

```

```

314 // 处理N(s)N(t)连接运算
315 Elem DFAbuilder::act_Join(Elem fir, Elem sec)
316 {
317     // 将fir的结束状态和sec的开始状态合并，将sec的边复制给fir，将fir返回
318     // 将sec总所有以StartState开头的边全部修改
319     /*
320     for (int i = 0; i < sec.edgeCount; ++i)
321     {
322         if (sec.edgeSet[i].startname == sec.startname)
323             sec.edgeSet[i].startname = fir.endname; // 该边e1的开始状态是N(t)的开始状态
324         else if (sec.edgeSet[i].endname == sec.endname)
325             sec.edgeSet[i].endname = fir.endname; // 该边e2的结束状态是N(t)的起始状态
326     }
327     sec.startname = fir.endname;
328     elem_copy(fir, sec);
329     // 将fir的结束状态更新为sec的结束状态
330     fir.endname = sec.endname;
331     */
332     Edge edge;
333     edge.startname = fir.endname;
334     edge.endname = sec.startname;
335     edge.tranSymbol = "#";
336     fir.edgeCount++;
337     fir.edgeSet.push_back(edge);
338     elem_copy(fir, sec);
339     fir.endname = sec.endname;
340     return fir;
341 }
342

```

```

344 // 处理闭包运算
345 Elem DFAbuilder::act_Closure(Elem elem)
346 {
347     Elem newelem;
348     newelem.edgeCount = 0;
349     Edge edge1, edge2, edge3, edge4;
350
351     // 获得新状态节点
352     Node startnode = new_node();
353     Node endnode = new_node();
354
355     // e1
356     edge1.startname = startnode.nodename;
357     edge1.endname = endnode.nodename;
358     edge1.tranSymbol = "#";
359
360     // e2
361     edge2.startname = elem.endname;
362     edge2.endname = elem.startname;
363     edge2.tranSymbol = "#";
364
365     // e3
366     edge3.startname = startnode.nodename;
367     edge3.endname = elem.startname;
368     edge3.tranSymbol = "#";
369
370     // e4
371     edge4.startname = elem.endname;
372     edge4.endname = endnode.nodename;
373     edge4.tranSymbol = "#";
374
375     // 构建单元
376     elem_copy(newelem, elem);
377
378     // 将新构建的四条边加入edgeSet
379     newelem.edgeSet.push_back(edge1);
380     newelem.edgeSet.push_back(edge2);
381     newelem.edgeSet.push_back(edge3);
382     newelem.edgeSet.push_back(edge4);
383
384     newelem.edgeCount += 4;
385     // 构建newelem的起始状态和结束状态
386     newelem.startname = startnode.nodename;
387     newelem.endname = endnode.nodename;
388
389     return newelem;
390 }

```

```

392 // 处理正闭包运算
393 Elem DFAbuilder::act_pClosure(Elem elem)
394 {
395     Elem newelem;
396     newelem.edgeCount = 0;
397     Edge edge1, edge2, edge3;
398
399     // 获得新状态节点
400     Node startnode = new_node();
401     Node endnode = new_node();
402
403     // e1
404     edge1.startname = elem.endname;
405     edge1.endname = endnode.nodename;
406     edge1.tranSymbol = "#";
407
408     // e2
409     edge2.startname = elem.endname;
410     edge2.endname = elem.startname;
411     edge2.tranSymbol = "#";
412
413     // e3
414     edge3.startname = startnode.nodename;
415     edge3.endname = elem.startname;
416     edge3.tranSymbol = "#";
417
418     // 构建单元
419     elem_copy(newelem, elem);
420
421     // 将新构建的三条边加入edgeSet
422     newelem.edgeSet.push_back(edge1);
423     newelem.edgeSet.push_back(edge2);
424     newelem.edgeSet.push_back(edge3);
425
426     newelem.edgeCount += 3;
427     // 构建newelem的起始状态和结束状态
428     newelem.startname = startnode.nodename;
429     newelem.endname = endnode.nodename;
430
431     return newelem;
432 }
433
434 // 处理可选运算
435 Elem DFAbuilder::act_Choosable(Elem fir, Elem sec)
436 {
437     Edge edge1, edge2;
438     edge1.startname = fir.endname;
439     edge1.endname = sec.startname;
440     edge1.tranSymbol = "#";
441     edge2.startname = fir.startname;
442     edge2.endname = fir.endname;
443     edge2.tranSymbol = "#";
444     fir.edgeCount+=2;
445     fir.edgeSet.push_back(edge1);
446     fir.edgeSet.push_back(edge2);
447     elem_copy(fir, sec);
448     fir.endname = sec.endname;
449     return fir;
450 }
451

```

图 4-2-9 到 图 4-2-13 为各个运算的处理

构建 NFA 的代码如下：

```

454 Elem DFAbuilder::buildNFA(string postfix, vector<string> subre_name)
455 {
456     stack<Elem> elem_st;
457     Elem elem, fir, sec;
458     string part = "";
459     char c;
460
461     map<int, int> sub_pos;
462     if (subre_name.size() > 1)
463     {
464         for (string it : subre_name)
465         {
466             int last = 0;
467             string tmp = postfix;
468             while (tmp.find(it) != string::npos)
469             {
470                 sub_pos.insert({ last + tmp.find(it), it.size() });
471                 last = tmp.find(it) + it.size();
472                 tmp = tmp.substr(last);
473             }
474         }
475     }
476
477     size_t i = 0;
478     while(i < postfix.size())
479     {
480         part = "";
481         c = postfix[i];
482
483         auto it = sub_pos.find(i);
484         if (it != sub_pos.end())
485         {
486             string sub_posfix = postfix.substr(it->first, it->second);
487             part = sub_posfix;
488             i += it->second;
489             --i;
490         }
491         else if (isalnum(c))
492             part = c;
493         else if (op_priority.find(c) == op_priority.end())
494         {
495             part = c + string(1, postfix[i + 1]) + string(1, postfix[i + 2]) + string(1, postfix[i + 3]);
496             if (find(symbol.begin(), symbol.end(), part) != symbol.end())
497                 i += 3;
498             else
499             {
500                 part = c + string(1, postfix[i + 1]) + string(1, postfix[i + 2]);
501                 if (find(symbol.begin(), symbol.end(), part) != symbol.end())
502                     i += 2;
503                 else
504                 {
505                     part = c + string(1, postfix[i + 1]);
506                     if (find(symbol.begin(), symbol.end(), part) != symbol.end())
507                         ++i;
508                     else
509                         part = c;
510                 }
511             }
512         }
513     }
514     ++i;

```

```

516 switch (c) {
517 case '*':
518     fir = elem_st.top();
519     elem_st.pop();
520     elem = act_Closure(fir);
521     elem_st.push(elem);
522     break;
523 case '+':
524     fir = elem_st.top();
525     elem_st.pop();
526     elem = act_pClosure(fir);
527     elem_st.push(elem);
528     break;
529 case '?':
530     sec = elem_st.top();
531     elem_st.pop();
532     fir = elem_st.top();
533     elem_st.pop();
534     elem = act_Choosable(fir, sec);
535     elem_st.push(elem);
536     break;
537 case '|':
538     sec = elem_st.top();
539     elem_st.pop();
540     fir = elem_st.top();
541     elem_st.pop();
542     elem = act_Unit(fir, sec);
543     elem_st.push(elem);
544     break;
545 case '.':
546     sec = elem_st.top();
547     elem_st.pop();
548     fir = elem_st.top();
549     elem_st.pop();
550     elem = act_Join(fir, sec);
551     elem_st.push(elem);
552     break;
553 default:
554     elem = act_Elem(part);
555     elem_st.push(elem);
556 }
557 }
558 elem = elem_st.top();
559 elem_st.pop();
560 nodenum = 0;
561 return elem;
562 }

```

图 4-2-14、图 4-2-15 NFA 图构建程序的代码

完成 NFA 图的构建后，基于 NFA 图构建的关系表，可以完成 DFA 图的构建，其代码如下：

```
565 // 检查DFA状态是否在状态集合中，即dfastates中是否能找到targetstate
566 bool DFABuilder::isDFAStateInVector(const vector<DFAState>& dfastates, const DFAState& targetstate)
567 {
568     // 遍历dfastates中的每一个状态
569     for (const DFAState& state : dfastates)
570     // 若找到匹配的状态，则返回true
571     if (state.statename == targetstate.statename)
572         return true;
573     // 遍历完整个状态集合仍未找到匹配的状态，则返回false
574     return false;
575 }
576
577 // 检查转换边是否在边集合中，比如a->b是否已经在边集合中
578 bool DFABuilder::isTransitionInVector(DFAState dfastate, DFAState dfanextstate, string symbol, vector<DFATransition> dfatransitions)
579 {
580     // 遍历dfatransitions中的每条转换边
581     for (const DFATransition& transition : dfatransitions)
582     // 若找到，返回true
583     if (transition.fromstate.statename == dfastate.statename && transition.to.state.statename == dfanextstate.statename && transition.transitionsymbol == symbol)
584         return true;
585     // 若找不到，返回false
586     return false;
587 }
588
589 // 计算NFA状态的ε闭包，也即计算NFA图中从某一状态可通过ε转换到所有状态的集合（包括当前状态）
590 DFAState DFABuilder::eClosure(const set<string>& nfastates, Elem nfa)
591 {
592     DFAState eClosureState;
593     eClosureState.nfaStates = nfastates;
594     stack<string> state_st;
595
596     // 初始化栈，将初始状态入栈，一开始nfastate中只有NFA_elem.startname
597     for (const string& nfastate_name : nfastates)
598         state_st.push(nfastate_name);
599
600     while (!state_st.empty())
601     {
602         string cur_state = state_st.top();
603         state_st.pop();
604
605         // 遍历NFA图的边
606         for (int i = 0; i < nfa.edgeCount; ++i)
607         {
608             Edge cur_edge = nfa.edgeSet[i];
609
610             // 如果边的起始状态是当前状态，并且边的转换符号为“ε”（也即ε转换边），则将目标状态加入ε闭包
611             if (cur_edge.startname == cur_state && cur_edge.transition == "ε")
612             // 检查目标状态是否已经在ε闭包中，避免重复添加
613             if (eClosureState.nfaStates.find(cur_edge.endname) == eClosureState.nfaStates.end())
614             {
615                 eClosureState.nfaStates.insert(cur_edge.endname);
616                 state_st.push(cur_edge.endname);
617             }
618         }
619     }
620
621     // 为ε闭包分配一个唯一的名称
622     for (const string& nfastate_name : eClosureState.nfaStates)
623         eClosureState.statename += nfastate_name + " ";
624
625     return eClosureState;
626 }
```

```

629 // move函数, 用于计算DFA状态的转移, 即获取通过非空转移到下一节点
630 DFAState DFAbuilder::move(const DFAState& dfastate, string transitionsymbol, Elem nfa)
631 {
632     DFAState nextstate;
633     // 遍历DFAState中的每一个NFA状态
634     for (const string& nfastate_name : (dfastate.nfaStates))
635     // 在这里遍历所有NFA状态的边
636     for (int i = 0; i < nfa.edgeCount; ++i)
637     {
638         Edge cur_edge = nfa.edgeSet[i];
639
640         // 若边的起始状态就是当前状态, 且边的转换符号等于输入符号, 则将目标状态加入nextstate
641         if (cur_edge.startname == nfastate_name && cur_edge.tranSymbol == transitionsymbol && cur_edge.tranSymbol != "#")
642             nextstate.nfaStates.insert(cur_edge.endname);
643     }
644
645     // 为nextstate分配一个唯一的名称
646     for (const string& nfastate_name : nextstate.nfaStates)
647         nextstate.statename += nfastate_name + ' ';
648
649     return nextstate;
650 }
651
652
653
654
655 void DFAbuilder::buildDFA(Elem& NFA_elem, vector<DFAState>& dfaStates, vector<DFATransition>& dfaTransitions)
656 {
657     // 初始化DFA状态集合和转换关系
658     set<string> nfa_initial_StateSet;
659     nfa_initial_StateSet.insert(NFA_elem.startname);
660     // 计算NFA初始状态的ε闭包, 得到DFA图的第一个节点(起点)
661     DFAState dfa_initial_State = eClosure(nfa_initial_StateSet, NFA_elem);
662     dfaStates.push_back(dfa_initial_State);
663
664     // 开始构建DFA
665     for (size_t i = 0; i < dfaStates.size(); ++i)
666     {
667         DFAState dfastate = dfaStates[i];
668         for (int j = 0; j < NFA_elem.edgeCount; ++j)
669         {
670             string symbol = NFA_elem.edgeSet[j].tranSymbol;
671             DFAState nextstate = move(dfastate, symbol, NFA_elem);
672             // 计算move操作后的ε闭包
673             DFAState dfanextstate = eClosure(nextstate.nfaStates, NFA_elem);
674
675             if (!nextstate.nfaStates.empty())
676             {
677                 // 若下一状态不为空, 且在DFA状态集合中还未添加, 则加入DFA状态集合
678                 if (!isDFAStateInVector(dfaStates, dfanextstate))
679                     dfaStates.push_back(dfanextstate);
680                 // 对于边也要去重, 因为等于a的边可能会遍历到两次
681                 // 若当前边在DFA转换关系中还未添加, 则加DFA转换关系
682                 if (!isTransitionInVector(dfastate, dfanextstate, symbol, dfaTransitions))
683                     dfaTransitions.push_back({ dfastate, dfanextstate, symbol });
684             }
685         }
686     }
687 }
688
689
690
691
692 void DFAbuilder::buildDFA(Elem& NFA_elem, vector<DFAState>& dfaStates, vector<DFATransition>& dfaTransitions)
693 {
694     // 初始化DFA状态集合和转换关系
695     set<string> nfa_initial_StateSet;
696     nfa_initial_StateSet.insert(NFA_elem.startname);
697     // 计算NFA初始状态的ε闭包, 得到DFA图的第一个节点(起点)
698     DFAState dfa_initial_State = eClosure(nfa_initial_StateSet, NFA_elem);
699     dfaStates.push_back(dfa_initial_State);
700
701     // 开始构建DFA
702     for (size_t i = 0; i < dfaStates.size(); ++i)
703     {
704         DFAState dfastate = dfaStates[i];
705         for (int j = 0; j < NFA_elem.edgeCount; ++j)
706         {
707             string symbol = NFA_elem.edgeSet[j].tranSymbol;
708             DFAState nextstate = move(dfastate, symbol, NFA_elem);
709             // 计算move操作后的ε闭包
710             DFAState dfanextstate = eClosure(nextstate.nfaStates, NFA_elem);
711
712             if (!nextstate.nfaStates.empty())
713             {
714                 // 若下一状态不为空, 且在DFA状态集合中还未添加, 则加入DFA状态集合
715                 if (!isDFAStateInVector(dfaStates, dfanextstate))
716                     dfaStates.push_back(dfanextstate);
717                 // 对于边也要去重, 因为等于a的边可能会遍历到两次
718                 // 若当前边在DFA转换关系中还未添加, 则加DFA转换关系
719                 if (!isTransitionInVector(dfastate, dfanextstate, symbol, dfaTransitions))
720                     dfaTransitions.push_back({ dfastate, dfanextstate, symbol });
721             }
722         }
723     }
724 }
725
726
727
728

```

图 4-2-16、图 4-2-17、图 4-2-18 NFA 图构建程序的代码

DFA 最小化代码如下:

```

690 void DFABuilder::minimizeDFA(vector<string>& endstates, vector<string>& notendstates, vector<DFATransition>& dfaTransitions)
691 {
692     // 初始化分区 (接受状态组与非接受状态组)
693     vector<set<string>> partitions;
694     // 结束状态组
695     partitions.push_back(set<string>(endstates.begin(), endstates.end()));
696     // 非结束状态组
697     partitions.push_back(set<string>(notendstates.begin(), notendstates.end()));
698
699     // 记录状态所属的分区编号
700     unordered_map<string, int> statePartition;
701     for (const auto& state : endstates) statePartition[state] = 0;
702     for (const auto& state : notendstates) statePartition[state] = 1;
703
704     // 构建状态转移表 (格式为: 源状态——转移条件——>目标状态)
705     // 将DFA的转移关系转换为双层哈希表, 快速查询任意状态在特定转移条件下的转移目标
706     unordered_map<string, unordered_map<string, string>> transitionTable;
707     for (const auto& trans : dfaTransitions)
708         transitionTable[trans.fromstate.statename][trans.transitionsymbol] = trans.tostate.statename;
709
710     // Hopcroft算法迭代划分
711     // 通过队列管理待处理的分区, 初始处理所有非接受状态和接受状态, push进去的为分区序号
712     queue<int> processQueue;
713     processQueue.push(0); // 接受状态分区
714     processQueue.push(1); // 非接受状态分区
715
716     while (!processQueue.empty())
717     {
718         // 获取当前分区序号, 确认当前处理的分区
719         int currentPart = processQueue.front();
720         processQueue.pop();
721
722         // 获取当前分区的所有符号
723         // 通过符号集合, 收集当前分区中所有状态的所有输入符号 (转换条件), 用于后续划分
724         set<string> symbols;
725         for (const auto& state : partitions[currentPart])
726             for (const auto& item : transitionTable[state])
727                 symbols.insert(item.first);
728
729         // 对每个符号进行切割
730         for (const auto& symbol : symbols)
731         {
732             // 根据符号转移后的目标分区对当前分区进行分组, 若两个状态在相同符号下转移到不同分区, 则需要分割新的分区
733             unordered_map<int, set<string>> splitMap;
734
735             // 根据符号转移后的目标分区进行分组
736             for (const auto& state : partitions[currentPart])
737             {
738                 if (transitionTable[state].count(symbol))
739                 {
740                     string target = transitionTable[state][symbol];
741                     int targetPart = statePartition[target];
742                     splitMap[targetPart].insert(state);
743                 }
744                 // splitMap[-1]表示当前状态在当前符号 (转移条件) 下无转移
745                 else splitMap[-1].insert(state);
746             }
747
748             // 如果产生新的分区, 则动态更新现有分区
749             // 通过不断细化分区直至无法分割, 得到最终分区结果
750             if (splitMap.size() > 1)
751             {
752                 partitions.erase(partitions.begin() + currentPart);
753                 int newIndex = currentPart;
754                 for (const auto& item : splitMap)
755                 {

```



```

822         return false;
823     }
824 }
825     return true;
826 },
827     minimizedEnd.end()
828 );
829
830
831 // 更新结果
832 endstates = minimizedEnd;
833 notendstates = minimizedNotEnd;
834 dfaTransitions = minimizedTrans;
835 }

```

4-2-2 展示处理结果

将转换关系写入 dot 文件并转换图片的代码如下：

```

838 void generateDotFile_NFA(const Elem& nfa, string filename) {
839     ofstream dotFile(filename + "NFAgraph.dot");
840
841     if (dotFile.is_open()) {
842         dotFile << "digraph NFA {\n";
843         dotFile << "    rankdir=LR; // 横向布局\n\n";
844         dotFile << "    node [shape = circle]; // 状态节点\n\n";
845
846         dotFile << nfa.endname << " [shape=doublecircle];\n";
847         // 添加 NFA 状态
848         dotFile << "    " << nfa.startname << " [label=\"Start State: \" << nfa.startname << "\"]; \n";
849         dotFile << "    " << nfa.endname << " [label=\"End State: \" << nfa.endname << "\"]; \n";
850
851         // 添加 NFA 转移
852         for (int i = 0; i < nfa.edgeCount; i++) {
853             const Edge& currentEdge = nfa.edgeSet[i];
854             dotFile << "    " << currentEdge.startname << " -> " << currentEdge.endname << " [label=\"" << currentEdge.transSymbol << "\"]; \n";
855         }
856
857         dotFile << " \n";
858
859         dotFile.close();
860         std::cout << "NFA DOT file generated successfully.\n";
861     }
862     else {
863         cerr << "Unable to open NFA DOT file.\n";
864     }
865
866     wstring wfilename(filename.begin(), filename.end());
867     wstring dotCmd_tmp = L"..\\packages\\Graphviz.2.38.0.2\\dot.exe -Tsvg "
868         + wfilename + L"NFAgraph.dot -o " + wfilename + L"NFA.svg";
869
870     wstring dotCmd = dotCmd_tmp;
871
872     STARTUPINFO si = { sizeof(si) };
873     PROCESS_INFORMATION pi;
874     bool success = CreateProcessW(
875         NULL,
876         &dotCmd[0], // 命令行字符串
877         NULL,
878         NULL,
879         FALSE,
880         0,
881         NULL,
882         NULL,
883         &si,
884         &pi
885     );
886
887     wstring dotCmd_tmp_1 = L"..\\packages\\Graphviz.2.38.0.2\\dot.exe -Tbmp "
888         + wfilename + L"NFAgraph.dot -o " + wfilename + L"NFA.bmp";
889
890     wstring dotCmd_1 = dotCmd_tmp_1;
891
892     STARTUPINFO si_1 = { sizeof(si_1) };
893     PROCESS_INFORMATION pi_1;
894     bool success_1 = CreateProcessW(
895         NULL,
896         &dotCmd_1[0], // 命令行字符串
897         NULL,
898         NULL,
899         FALSE,
900         0,
901         NULL,
902         NULL,
903         &si_1,
904         &pi_1
905     );

```

```

903     &si_l,
904     &pi_l
905 );
906
907
908
909 //生成DFA的dot文件
910 void generateDotFile_DFA(vector<string>& endstates, vector<DFAState>& dfaStates, vector<DFATransition>& dfaTransitions, const Elem& nfa, string filename) {
911     vector<string> notendstates;
912     // 打开名为 "dfa_graph.dot" 的文件流
913     ofstream dotFile(filename + "DFAgraph.dot");
914
915     // 如果文件流成功打开
916     if (dotFile.is_open()) {
917         // 写入 DOT 文件的头部信息
918         dotFile << "digraph DFA {\n";
919         dotFile << "rankdir=LR; // 横向右局\n\n";
920         dotFile << "node [shape = circle]; // 初始状态\n\n";
921         for (DFAState dfaState : dfaStates)
922             if (dfaState.stateName.find(nfa.endname) != string::npos)
923                 dotFile << "\"" << dfaState.stateName << "\" [shape = doublecircle];\n";
924
925         // 添加DFA状态
926         for (const auto& state : dfaStates) {
927             dotFile << " " << state.stateName << ";\n";
928             dotFile << " [label=\"" << state.stateName << "\n";
929             if (state.stateName == dfaStates.front().stateName) dotFile << "\n(startState)";
930             if (state.stateName == dfaStates.back().stateName || state.stateName.find(nfa.endname) != string::npos) {
931                 dotFile << "\n(endState)";
932                 endstates.push_back(state.stateName);
933             }
934             else notendstates.push_back(state.stateName);
935             dotFile << "];\n";
936         }
937         dotFile << "\n";
938
939         // 添加DFA转移
940         for (const auto& transition : dfaTransitions) {
941             dotFile << " " << transition.fromState.stateName << " --> " << transition.toState.stateName << " [label=\"" << transition.transitionSymbol << "\n";
942         }
943
944         // 写入 DOT 文件的尾部信息
945         dotFile << ";\n";
946
947         // 关闭文件流并输出成功信息
948         dotFile.close();
949         std::cout << "DFA DOT file generated successfully.\n";
950
951     }
952     else {
953         // 输出错误信息，表示无法打开 DOT 文件
954         std::cerr << "Unable to open DOT file.\n";
955     }
956
957     wstring wfilename(filename.begin(), filename.end());
958     wstring dotCmd_tmp = L"\".\\packages\\Graphviz.2.38.0.2\\dot.exe\" -Tsvg \""
959         + wfilename + L"DFAgraph.dot\" -o \"" + wfilename + L"DFA.svg\"";
960
961     wstring dotCmd = dotCmd_tmp;
962
963     STARTUPINFO si = { sizeof(si) };
964     PROCESS_INFORMATION pi;
965     bool success = CreateProcessW(
966         NULL,
967         dotCmd[0], // 命令行字符串
968         NULL,
969         NULL,
970         NULL,
971         0,
972         NULL,
973         NULL,
974         si,
975         pi
976     );
977 }

```

词法分析程序代码生成代码如下：

```

1134 string code_generate(string token_name, vector<string>& endstates, vector<DFATransition>& dfaTransitions)
1135 {
1136     map<pair<string, string>, string> transition_table;
1137     map<string, string> token_map;
1138     for (auto& it : dfaTransitions)
1139         transition_table.insert({ {it.fromstate.statename, it.transitionsymbol}, it.tostate.statename });
1140     for (auto& it : endstates)
1141         token_map.insert({ it, token_name });
1142
1143     string code = R"(
1144 #include <iostream>
1145 #include <string>
1146 #include <unordered_map>
1147 using namespace std;
1148
1149 // DFA states transition table
1150 unordered_map<int, unordered_map<char, int>> transitions = {
1151 }";
1152     // 生成转移表代码
1153     map<string, map<string, string>> grouped;
1154     for (auto& entry : transition_table) {
1155         const auto& key_pair = entry.first; // 原始键 (pair<int, char>)
1156         const string state = key_pair.first; // 分解键的第一个部分
1157         const string input_char = key_pair.second; // 分解键的第二部分
1158         const string target_state = entry.second; // 值 (目标状态)
1159         grouped[state][input_char] = target_state;
1160     }
1161
1162     for (auto& outer_entry : grouped) {
1163         const string state = outer_entry.first;
1164         const auto& trans = outer_entry.second;
1165
1166         code += "    { " + state + ", {\n";
1167
1168         for (auto& inner_entry : trans) {
1169             const string ch = inner_entry.first;
1170             const string target = inner_entry.second;
1171
1172             code += "        { " + ch + ", " + target + " },\n";
1173         }
1174
1175         code += "    },\n";
1176     }
1177
1178     // 生成接受状态映射
1179     code += "unordered_map<int, string> accept_tokens = {\n";
1180     for (auto& entry : token_map) {
1181         const string state = entry.first;
1182         const string token = entry.second;
1183         code += "    { " + state + ", \"" + token + "\" },\n";
1184     }
1185     code += "};\n";
1186
1187     // 生成词法分析函数
1188     code += R"(
1189 string lex_analyze(const string& input) {
1190     int current_state = 0; // the start state
1191     string token;
1192     string result;
1193
1194     for (char c : input) {
1195         auto& state_map = transitions[current_state];
1196         auto it = state_map.find(c);
1197
1198         if (it == state_map.end()) {
1199             // deal with the error
1200             cerr << "Lexical error at: " << c << endl;

```

```

1200         return "";
1201     }
1202
1203     current_state = it->second;
1204     token += c;
1205
1206     // check the accepted states
1207     if (accept_tokens.count(current_state)) {
1208         result += accept_tokens[current_state] + " ";
1209         token.clear();
1210     }
1211 }
1212
1213 // deal with the last token
1214 if (!token.empty() && accept_tokens.count(current_state)) {
1215     result += accept_tokens[current_state];
1216 }
1217
1218 return result;
1219 }
1220
1221 int main() {
1222     string input;
1223     cout << "Input string: ";
1224     getline(cin, input);
1225     cout << "Tokens: " << lex_analyze(input) << endl;
1226     return 0;
1227 }
1228 // return code;
1229 }
1230

```

5. 系统测试

5-1 系统启动

由于已经生成了 Release 版本，直接双击可执行程序即可打开：



5-2 数据测试

根据测试数据中的内容进行各项测试，其结果均显示正确，结果放在测试结果文件夹中

四、实验总结（心得体会）

本次实验要求设计并实现一个图形界面 C++ 应用程序，能够将正则表达式转换为 NFA、DFA 以及最小化 DFA，并最终生成词法分析程序。通过此次实验，我深入理解了正则表达式与有限自动机之间的转换原理，掌握了 DFA 最小化算法的实现方法，同时熟悉了图形化用户界面开发的关键技术。

在实验过程中，我首先进行了系统设计，将系统划分为用户交互界面、正则表达式解析模块、NFA 构造模块、DFA 转换模块、DFA 最小化模块和代码生成模块。用户交互界面提供了直观的窗口，包含文本编辑区域、按钮和表格视图，允许用户输入正则表达式并查看转换结果。正则表达式解析模块负责预处理输入的正则表达式，处理中括号、正闭包和可选符号等，并将其转换为后缀表达式。NFA 构造模块使用栈操作从后缀表达式构建 NFA，支持空转移和动态状态管理。DFA 转换模块通过子集构造法将 NFA 转换为 DFA，并计算空闭包和状态转移闭包。DFA 最小化模块使用状态划分算法对 DFA 进行最小化，优化状态转移表。代码生成模块将最小化 DFA 转换为匹配代码，并提供代码查看窗口。

通过本次实验，我不仅提升了编程实践能力，还加深了对编译原理中词法分析部分的理解。总的来说，这次实验为我后续学习和开发更复杂的软件项目奠定了坚实的基础

五、参考文献：

正则表达式 -> NFA -> DFA -> 最小化 DFA 图并生成词法分析程序：

https://blog.csdn.net/weixin_50549897/article/details/134065599